ESP32 Dual Axis Motion Control Walkthrough & Guide

Bu kılavuz; oluşturulan kodların Arduino IDE ile derlenmesini, ESP32 ve DRV8825 donanım bağlantılarını, motor parametrelerinin kalibrasyonunu ve sistemin test prosedürünü detaylı şekilde açıklar.

---

1. Donanım Bağlantı Şeması

Aşağıdaki şemada ESP32 DevKit V1, iki adet DRV8825 sürücü kartı ve limit switch'lerin bağlantı yapısı gösterilmiştir:

```
graph TD
    ESP32[ESP32 DevKit V1]
    DRV1[DRV8825 - Motor 1 X Ekseni]
    DRV2[DRV8825 - Motor 2 Y Ekseni]
    SW1[Home Switch 1 - Motor 1]
    SW2[Home Switch 2 - Motor 2]

    %% ESP32 to DRV1 Pins
    ESP32 -->|GPIO25| DRV1[STEP Pin]
    ESP32 -->|GPIO26| DRV1[DIR Pin]
    ESP32 -->|GPIO27| DRV1[ENABLE Pin]
    ESP32 -->|GPIO32| SW1[Signal Pin]

    %% ESP32 to DRV2 Pins
    ESP32 -->|GPIO18| DRV2[STEP Pin]
    ESP32 -->|GPIO19| DRV2[DIR Pin]
    ESP32 -->|GPIO21| DRV2[ENABLE Pin]
    ESP32 -->|GPIO33| SW2[Signal Pin]

    %% Limit Switch Ground
    SW1 --> GND[GND Ortak]
    SW2 --> GND
```

---

2. DRV8825 Sürücü Bağlantıları

Her bir DRV8825 sürücü entegresi için pin eşleştirmesi şu şekildedir:

DRV8825 Pin	Bağlanacağı Yer	Açıklama
VMOT	24V DC (+)	Sürücü Motor Güç Girişi
GND (Power)	24V DC (-)	Güç Kaynağı Toprak Hattı
SLP / RST	Birbirine Kısa Devre	RESET ve SLEEP pinleri HIGH seviyede olmalıdır (SLP ve RST pinlerini fiziksel olarak köprüleyin)
VDD	5V DC (+)	Sürücü Mantıksal Beslemesi
GND (Logic)	ESP32 GND (-)	Mantıksal Toprak Hattı
STEP	Motor 1 için GPIO25 / Motor 2 için GPIO18	Adım Sinyali Girişi
DIR	Motor 1 için GPIO26 / Motor 2 için GPIO19	Yön Sinyali Girişi (HIGH = Bir Yön, LOW = Diğer Yön)
ENABLE	Motor 1 için GPIO27 / Motor 2 için GPIO21	Aktif LOW etkinleştirme pini (LOW = Motor devrede, HIGH = Motor serbest)
M0, M1, M2	3.3V / 5V (+) (HIGH)	1/16 Adım Modu (DRV8825 için hepsi HIGH = 1/16 Microstep)
A1, A2, B1, B2	NEMA14 Step Motor Bobinleri	Step motorun 4 kablosu (A ve B faz çiftleri)

```
<div style="border-left: 5px solid #ef4444; background-color: #fef2f2; color: #991b1b; p
    <strong>IMPORTANT</strong><br>> **DRV8825 Akım Sınırlaması (Vref):**
```

> *NEMA14 motorları çalıştırmadan önce DRV8825 üzerindeki trimpotu çevirerek akım sınırını ayarlayın. Akım formülü:* `Vref = Akım Sınırı (A) * 0.5`.
> *Örneğin 1A çeken bir motor için trimpot üzerindeki Vref voltajı 0.5V olarak ayarlanmalıdır.*

---

3. Home Switch (Limit) Bağlantıları

Home switch'leri mekanik olarak normalde açık (Normally Open - NO) tipte seçilmelidir.
- **Motor 1 Switch:** Bir pini **GPIO32**'ye, diğer pini ise **GND**'ye bağlanır.
- **Motor 2 Switch:** Bir pini **GPIO33**'ye, diğer pini ise **GND**'ye bağlanır.

```
<div style="border-left: 5px solid #3b82f6; background-color: #eff6ff; color: #1e40af; p
    <strong>NOTE</strong><br>> Yazılımda GPIO32 ve GPIO33 pinleri `INPUT_PULLUP` olarak
</div>
```

---

4. Güç Dağıtımı (Power Distribution)

Sistemde iki farklı voltaj kullanılmaktadır:
1. **24V DC Güç Kaynağı:**
- Motorların güç beslemesidir. DRV8825'lerin VMOT(+) ve GND(-) pinlerine doğrudan bağlanır.
- 24V hattına yakın bir yere gürültüyü sönümlemek amacıyla en az **100uF elektrolitik kondansatör** (VMOT ve GND arasına kutuplara dikkat edilerek) paralel bağlanmalıdır.
2. **5V DC Regüle Besleme:**
- ESP32'nin **5V / VIN** pinine ve DRV8825'lerin **VDD** pinlerine bağlanır.
3. **GND Birleştirme:**
- 24V güç kaynağının GND hattı, 5V regülatörün GND hattı ve ESP32'nin GND pinleri **birbirine mutlaka ortak bağlanmalıdır**. Ortak şase yapılmazsa motorlar kararsız döner veya sürücüler zarar görebilir.

---

5. Kütüphane Kurulumu

Projenin derlenebilmesi için Arduino IDE'de **Library Manager (Kütüphane Yöneticisi)** üzerinden şu kütüphanelerin kurulması gerekir:

1. **FastAccelStepper** (yazan: *Erich Styger*)
2. **ArduinoJson** (yazan: *Benoit Blanchon*) - *v6 veya v7 sürümleri ile tam uyumludur.*

3. **ESPAsyncWebServer** ve **AsyncTCP**
4. *Not:* ESPAsyncWebServer resmi Arduino kütüphane havuzunda olmayabilir. Kurulum için ZIP olarak indirilip kütüphane klasörüne eklenmesi gerekebilir:
   - ESPAsyncWebServer GitHub
   - AsyncTCP GitHub

---

6. Derleme Ayarları

Arduino IDE 2.x üzerinde:
1. **Board (Kart):** `ESP32 Dev Module` veya `DOIT ESP32 DEVKIT V1` seçin.
2. **Flash Frequency:** `80MHz`
3. **Partition Scheme:** `Default 4MB with spiffs (1.2MB APP / 1.5MB SPIFFS)` veya `Minimal SPIFFS (Large APPS)` seçin. Web arayüzü kod içinde gömülü (`index_html.h`) olduğu için SPIFFS dosyası yükleme ihtiyacı yoktur.
4. **Upload Speed:** `921600` veya `115200`.

---

7. İlk Devreye Alma Prosedürü

1. Motorların güç bağlantılarını (24V) yapmadan önce sadece ESP32'yi USB ile bilgisayara bağlayın.
2. Kodları ESP32'ye yükleyin.
3. Seri port ekranını (`115200 baud`) açın. "ESP32 Dual Axis Stepper Motor System Initializing..." yazısını görün.
4. Bilgisayarınızdan veya telefonunuzdan Wi-Fi ağlarını taratın. **SSID: ESP32-MOTOR** ismini görüp **12345678** şifresiyle bağlanın.
5. Tarayıcınızdan `http://192.168.4.1` adresine gidin. Arayüzün geldiğini ve "WebSocket: Bağlandı" durumunu teyit edin.
6. 24V motor gücünü açın.
7. Arayüzden önce "Güç Ver (EN)" butonuna basarak sürücülerin aktif olduğunu (sesinden veya elinizle motor milini çeviremediğinizden) teyit edin.

---

8. Test Prosedürü

1. **Jog Testi:** Arayüzde Motor 1 için "+10" ve "-10" butonlarına sırasıyla basarak motorun ileri ve geri adımladığını teyit edin.

2. **Stop Testi:** Motor dönerken arayüzdeki kırmızı "DUR" butonuna basın ve motorun rampalı şekilde durduğunu izleyin.
3. **Acil Durdurma Testi:** İki motor birden hareket ederken en üstteki **ALL STOP (ACİL DUR)** butonuna basın. İki motorun da anında hareketini kestiğini gözlemleyin.
4. **Timeout Testi:** Motorlar hareketsiz kaldığında 10 saniye sonra sürücü enerjisinin kesildiğini (motor milinin serbest kaldığını) ve tekrar hareket komutu verildiğinde sürücünün otomatik olarak devreye girdiğini doğrulayın.

---

9. Soft Limit Kalibrasyonu

Yazılımsal limitler (Soft Limits), motorun mekanik sınırlara çarpmasını önlemek için adım bazında kontrol yapar.
1. Motorunuzu sıfır noktasına kadar elinizle veya jog tuşlarıyla getirin.
2. Arayüzden 📍 **0 NOKTASI YAP** butonuna basarak bu noktayı sıfırlayın.
3. Motoru güvenli bir şekilde hareket ettirebileceğiniz en son negatif ve pozitif sınırları belirleyin (Örnek: Min: -2000 step, Max: 80000 step).
4. Arayüzdeki **Motor Ayarları** sekmesinden bu değerleri "Yazılımsal Min Sınır" ve "Yazılımsal Max Sınır" alanlarına kaydedin.
5. Arayüzden bu sınırların dışına çıkan bir git komutu verin. Sistem hareket etmeyecek ve günlüğe "Hata: Motor yazılımsal sınır ihlali!" uyarısı yazacaktır.

---

10. Pulse/mm Hesaplama Örnekleri

Sistem girilen fiziksel parametrelerden step/birim katsayılarını otomatik hesaplar. Formül mantığı şu şekildedir:

A. Vidalı Mil (Lead Screw) Örneği

- **Step Angle (Adım Açısı):** 1.8° (Tur başına $360 / 1.8 = 200$ tam adım)
- **Microstep:** 1/16 (Tur başına $200 \times 16 = 3200$ mikrostep)
- **Hatve (Lead Screw Pitch):** 8 mm (Motor 1 tur döndüğünde 8 mm ilerler)

$$\text{Steps Per Rev} = \frac{360}{1.8} \times 16 = 3200 \text{ steps}$$

$$\text{Pulse Per mm} = \frac{\text{Steps Per Rev}}{\text{Pitch}} = \frac{3200}{8} = 400 \text{ steps/mm}$$

$$\text{1 Step Karşılığı mm} = \frac{1}{400} = 0.0025 \text{ mm}$$

B. Kayış & Kasnak (Belt & Pulley - GT2) Örneği

- **Step Angle:** 1.8° (Tur başına 200 adım)
- **Microstep:** 1/16 (Tur başına 3200 mikrostep)
- **Diş Hatvesi (Belt Pitch):** 2 mm (GT2 kayış)
- **Diş Sayısı (Pulley Teeth):** 20T (Kasnakta 20 diş var)

$$\text{Tur Başına Mesafe (Kasnak Çevresi)} = 2 \text{ mm} \times 20 = 40 \text{ mm}$$

$$\text{Pulse Per mm} = \frac{\text{Steps Per Rev}}{\text{Kasnak Çevresi}} = \frac{3200}{40} = 80 \text{ steps/mm}$$

$$\text{1 Step Karşılığı mm} = \frac{1}{80} = 0.0125 \text{ mm}$$

---

11. WebSocket Veri Yapısı (/ws)

WebSocket üzerinden her 200 ms'de bir gönderilen JSON telemetri paketi şu formatı taşır:

```
{
  "m1Pos": 12000,
  "m1PosU": 30.00,
  "m1Target": 20000,
  "m1TargetU": 50.00,
  "m1Dist": 8000,
  "m1DistU": 20.00,
  "m1Speed": 2000.0,
  "m1Accel": 1000.0,
  "m1Status": 2,
  "m1Sw": false,
  "m1En": true,
  "m1HomeOk": true,
  "m1Percent": 60,
  "m2Pos": 0,
  "m2PosU": 0.00,
  "m2Target": 0,
  "m2TargetU": 0.00,
  "m2Dist": 0,
  "m2DistU": 0.00,
  "m2Speed": 0.0,
  "m2Accel": 0.0,
  "m2Status": 1,
  "m2Sw": false,
  "m2En": false,
  "m2HomeOk": false,
  "m2Percent": 100,
  "m1Unit": 1,
  "m2Unit": 1,
  "wifiMode": "AP+STA",
  "wifiIP": "192.168.1.150",
  "uptime": 1420,
  "heap": 182344,
  "wdt": true,
  "log": "Hata: Motor 1 yazılımsal sınır ihlali!"
}
```


Not: `"Log"` alanı sadece sistemde yeni bir durum (hata, limit ihlali, home başlama vb.) meydana geldiğinde pakete eklenir.

---

12. Watchdog Çalışma Mantığı

ESP32 kilitlenmelerini ve çökmelerini önlemek için **Task Watchdog Timer (WDT)** devrededir:
1. Donanım seviyesinde çalışan WDT, 10 saniyelik bir geri sayım sayacı başlatır.
2. Loop döngüsünün her turunda `esp_task_wdt_reset()` fonksiyonu çağrılarak sayaç sıfırlanır (bekçi köpeği beslenir).
3. Eğer kodda sonsuz döngü, kilitlenme veya WiFi kütüphanelerinin sebep olduğu 10 saniyeyi aşan bir tıkanıklık oluşursa, WDT tetiklenir.
4. ESP32 donanımsal olarak kendini resetler, NVS'ten son koordinatları okur ve güvenli bir şekilde yeniden başlar.
5. Arayüzün sağ üst köşesindeki "WDT: AKTİF" ibaresi bu koruma sisteminin çalıştığını teyit eder.